

INSERT

```
INSERT INTO branch (name, location) VALUES
('A', 'Dhaka'), ('B', 'Chittagong');
```

PL/pgSQL Block & Control

```
DO $$
DECLARE
    n int := 5;    sum int := 0;    i int;
BEGIN
    FOR i IN 1..n-1 LOOP
        IF n % i = 0 THEN sum := sum + i; END IF;
    END LOOP;
    IF sum = n THEN
        RAISE NOTICE '%is a perfect number', n;
    ELSIF sum != n THEN
        RAISE NOTICE '%is NOT a perfect number', n;
    END IF;
END; $$
```

Stored Procedure

```
CREATE OR REPLACE PROCEDURE odd_even_counter(
    start_val INT, end_val INT)
LANGUAGE plpgsql AS $$
BEGIN
    -- procedure body here
END; $$
```

```
CALL odd_even_counter(24, 30);
```

Cursors

Implicit Cursor (FOR loop)

```
DO $$ DECLARE emp RECORD;
BEGIN
    FOR emp IN SELECT id, name FROM employees LOOP
        RAISE NOTICE 'ID:%,Name:%', emp.id,
        emp.name;
    END LOOP;
END $$;
```

Explicit Cursor

```
DO $$ DECLARE
    emp_cursor CURSOR FOR SELECT id, name FROM
    employees;
    emp_record RECORD;
BEGIN
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO emp_record;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'ID:%,Name:%',
            emp_record.id, emp_record.name;
    END LOOP;
    CLOSE emp_cursor;
END $$;
```

Cursor with UPDATE

```
DO $$ DECLARE
    cur CURSOR FOR SELECT emp_id, salary
    FROM employees WHERE dept_id=20 FOR UPDATE;
    r RECORD;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO r;
        EXIT WHEN NOT FOUND;
        UPDATE employees
        SET salary = salary * 1.10
```

```
WHERE CURRENT OF cur; -- points to current
row
END LOOP;
CLOSE cur;
END; $$;
```

Trigger

```
CREATE OR REPLACE FUNCTION set_created_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.created_at := CURRENT_TIMESTAMP;
    RETURN NEW;
END; $$;

CREATE TRIGGER trg_set_created_at
BEFORE INSERT ON students
FOR EACH ROW EXECUTE FUNCTION set_created_at();
```

Function (Loan Scheme)

```
CREATE OR REPLACE FUNCTION find_loan_scheme(p_user
    INT)
RETURNS INT AS $$
DECLARE
    total_trans NUMERIC;    rec RECORD;
BEGIN
    SELECT SUM(amount) INTO total_trans
    FROM transactions WHERE user_id = p_user;
    FOR rec IN
        SELECT * FROM loan_type ORDER BY min_trans
    DESC
    LOOP
        IF total_trans >= rec.min_trans THEN
            RETURN rec.scheme;
        END IF;
    END LOOP;
    RETURN NULL;
END; $$ LANGUAGE plpgsql;
```

```
SELECT find_loan_scheme(1);
```

Useful Snippets

```
-- Row count after DML
GET DIAGNOSTICS affected_rows = ROW_COUNT;
```

```
-- Insert from trigger context
INSERT INTO bill VALUES (NEW.ssn,
    EXTRACT(MONTH FROM CURRENT_DATE),
    EXTRACT(YEAR FROM CURRENT_DATE), 0);
RETURN NEW;
```

```
-- Date/time
CURRENT_DATE,    CURRENT_TIME,    CURRENT_TIMESTAMP
```

Sequence & ID Generation

```
CREATE SEQUENCE student_seq START 1;

CREATE OR REPLACE FUNCTION gen_id(
    adm DATE, d CHAR, p CHAR, s CHAR)
RETURNS VARCHAR AS $$
DECLARE yy TEXT; seq TEXT;
BEGIN
    yy := TO_CHAR(adm, 'YY');
    seq :=
        LPAD(nextval('public.student_seq')::TEXT, 2, '0');
    RETURN yy || '00' || d || p || s || seq;
END; $$ LANGUAGE plpgsql;
```

Recursive CTE

Task Dependencies

```
WITH RECURSIVE task_dependencies AS (  
    SELECT task_id, depends_on FROM project_tasks  
    WHERE depends_on IS NOT NULL  
    UNION  
    SELECT td.task_id, pt.depends_on  
    FROM task_dependencies td  
    JOIN project_tasks pt ON td.depends_on =  
    pt.task_id  
    WHERE pt.depends_on IS NOT NULL  
)  
  
SELECT task_id, depends_on FROM task_dependencies  
ORDER BY task_id, depends_on;
```

Employee Hierarchy

```
WITH RECURSIVE emp_dep AS (  
    SELECT emp_id, emp_name, manager_id, department,  
           1 AS level, CAST(emp_id AS TEXT) AS path  
    FROM employee_hierarchy WHERE manager_id IS NULL  
    UNION  
    SELECT e.emp_id, e.emp_name, e.manager_id,  
           e.department, h.level+1,  
           h.path || ',' || CAST(e.emp_id AS TEXT)  
    FROM employee_hierarchy e  
    JOIN emp_dep h ON e.manager_id = h.emp_id  
)  
  
subordinates AS (  
    SELECT h1.emp_id,  
           COUNT(h2.emp_id) AS total_subordinates  
    FROM emp_dep h1  
    LEFT JOIN emp_dep h2  
    ON h2.path LIKE h1.path || ',%'  
    GROUP BY h1.emp_id  
)  
  
SELECT h.*, s.total_subordinates  
FROM emp_dep h JOIN subordinates s ON  
    h.emp_id=s.emp_id  
ORDER BY h.level, h.emp_id;
```

Window Functions

```
DENSE_RANK() OVER (PARTITION BY department  
                    ORDER BY salary DESC) AS  
    dept_rank  
  
-- RANK: 1,2,2,4 | DENSE_RANK: 1,2,2,3  
-- ROW_NUMBER: 1,2,3,4  
  
salary - LAG(salary) OVER (ORDER BY salary)  
    AS salary_difference
```

String Functions

```
CONCAT(FIRST_NAME,'_',LAST_NAME) AS FULL_NAME  
INITCAP(name)    UPPER(name)    LOWER(email)  
LENGTH(email)  
LPAD(PHONE, 15, '*')    RPAD(PHONE, 15, '-')  
SUBSTR(EMAIL, 1, 4)  
SUBSTR(EMAIL, POSITION('@' IN EMAIL)+1) AS DOMAIN  
ROUND(AVG(discount_percent), 2)
```

Date & Regex

```
TO_CHAR(hire_date, 'DD-MM-YYYY')  
EXTRACT(YEAR FROM date_col)  
(transaction_date + INTERVAL '12_months')::DATE  
(DATE '2024-03-01' - transaction_date)  
  
-- Regex: extract 4-digit year from string  
WHERE CAST(REGEXP_REPLACE(date_str,  
    '.*(\d{4}).*', '\1') AS INTEGER) >= 2010;  
  
-- Email validation  
email !~ '^ [A-Za-z0-9._%+~@][A-Za-z0-9.\-~@]+  
          \. [A-Za-z]{2,}$'  
  
-- Title match & remove  
REGEXP_MATCHES(full_name,  
    '(Dr\.|Mr\.|Mrs\.|Ms\.|Prof\.|Rev\.|Sir\.)')  
REGEXP_REPLACE(full_name,  
    '(Dr\.|Mr\.|Mrs\.|Ms\.|Prof\.)\. *', '')  
WHERE full_name ~ '(Dr\.|Mr\.|Mrs\.|Ms\.)';
```

ROLLUP & COALESCE

```
GROUP BY ROLLUP(department, product)  
ORDER BY department NULLS LAST, product NULLS LAST;  
  
COALESCE(dept, '***_GRAND_TOTAL_***')  
-- returns second arg if first is NULL
```

Normalization (1NF → 3NF)

1NF: Each column must contain *atomic* (single) values.
2NF: 1NF + remove data depending on *part* of a composite key.
3NF: 2NF + remove columns that depend on *non-key* columns.

1NF: Flatten multi-valued cols

```
INSERT INTO Orders_1NF  
SELECT OrderID, OrderDate, CustomerID, CustomerName,  
    CustomerPhone, CustomerAddress,  
    Product2ID, Product2Name, Product2Price,  
    Product2Qty, PaymentMethod, PaymentStatus  
FROM Orders_Unnormalized WHERE Product2ID IS NOT  
    NULL  
UNION ALL  
SELECT OrderID, OrderDate, CustomerID, CustomerName,  
    CustomerPhone, CustomerAddress,  
    Product1ID, Product1Name, Product1Price,  
    Product1Qty, PaymentMethod, PaymentStatus  
FROM Orders_Unnormalized WHERE Product1ID IS NOT  
    NULL;
```

2NF: Junction / Composite PK

```
CREATE TABLE OrderItems_2NF (  
    OrderID    INT REFERENCES Orders_2NF(OrderID),  
    ProductID  INT REFERENCES  
    Products_2NF(ProductID),  
    ProductQty INT,  
    PRIMARY KEY (OrderID, ProductID)  
);
```

SERIAL Primary Key (3NF)

```
CREATE TABLE employees (  
    emp_id SERIAL PRIMARY KEY,  
    name    VARCHAR(100)  
);
```